



P11

Settlement Conservation in an Internal Task Market: Mechanism Design for Denying Duplicate Labor and Slashing Byzantine Agents

内部任务市场的结算守恒与拜占庭罚没机制

Settlement Conservation in an Internal Task Market: Mechanism Design for Denying Duplicate Labor and Slashing Byzantine Agents

Evidence grade: all figures come from CPU deterministic simulation (`reports7/matrix_scale_demo.json`, `evidence_grade=cpu-proto`; unit-test figures come from `tests7/test_v749_market.py` and the `market.py` code, verified), as **single-seed pilot point estimates**; the full-factorial simulation of strategy family $\times$ slashing rate $\times$ acceptance hit rate (≥ 1000 rounds/cell) has not been run, and the relevant positions keep [RESULT NEEDED]. Template-comparison positioning: this paper is the Stage B upgrade draft of the "UDOS Writing and Typesetting Specification v2." The empirical chapter unfolds as "experimental setup $\rightarrow$ comparison (unit-test ledger vs matrix clean/stressed) $\rightarrow$ sensitivity (analytical thresholds of slashing rate/acceptance hit rate) $\rightarrow$ validity threats"; conservation invariants Eqs.(1)–(3) are centrally numbered, and the clean/stressed two-level comparison table follows the NSA template's "same-budget two-level comparison" form.

Abstract (Structured, Four Parts)

[Background & Problem] As multi-agent systems scale from dozens to millions of agents, task allocation and contribution settlement move from an engineering detail to a governance problem. Without explicit settlement rules, resource consumption is a "dark ledger"; the first requirement of a ledger is not "optimal incentives" but "the books balance."

[Method] This paper reports the internal task market of UDOS v7.4.9 (`udos7/topology/market.py`): a settlement mechanism combining a quality floor, a value-for-money award rule, payment only to the unique accepted completer, and slashing of Byzantine results. We write the settlement rule as a priority short-circuit and formalize "balanced books" as a conservation invariant enforced under floating-point tolerance.

[Evidence & Results] The core invariant is `balance_sum = total_paid - slashed` with `total_paid  $\leq$  total_budget`. A mixed 3-task unit test measures: budget 24, paid 16, slashed 1, balance sum 15 ($=16-1$), holds 8, `conserved=true`; the end-to-end matrix integration (24 orders, 3 faulty nodes) is `conserved=true` in both clean and stressed runs, 24/24 settled by acceptance, retries produce no duplicate payment. Ten unit tests cover award, floor, load tie-break, unique payment, duplicate denial, slashing, conservation, and never-exceed-budget.

[Contribution] (1) Conservation is a theorem (guaranteed by settlement code structure), whereas incentive compatibility is an empirical approximation (depends on acceptance hit rate); (2) we provide a measured "settlement rule $\times$ behavior type $\rightarrow$ conservation/incentive" matrix; (3) we honestly scope it as internal accounting units with no real pricing game, no Sybil defense, and no financialization claim.

Keywords: mechanism design; task market; settlement conservation; Byzantine fault tolerance; multi-agent systems; incentive compatibility

1 Introduction

Multi-agent orchestration is moving from "a few agents collaborating" toward "hundreds, thousands, even million-agent matrices." When tasks flow among agents as work orders, three engineering problems immediately arise: who is qualified to take the task? By what rule is it awarded? After completion, on what basis is money paid, and how much? If these rules are implicit and rely on human coordination, once the system scales three things happen: **duplicate labor** (two agents do the same thing, resources wasted), **low-quality bid rushing** (the cheapest but laziest bidder always wins), and **false reporting** (submitting a result that is not qualified at all but demanding payment).

Why does this matter? Because in an agent system without explicit settlement rules, resource consumption is a "dark ledger": you do not know who did duplicate work, who took payment they should not have, or whether the total budget was spent or wasted. When the system has only 10 agents, human coordination can hold up; at 1000, 1 million agents, there is no governance without a ledger. The first requirement of a ledger is not "optimal incentives" but "the books balance" — money is neither created nor destroyed out of thin air, and every payment traces to an accepted work order. This paper formalizes this most basic engineering requirement as a conservation invariant and proves it always holds in code structure.

This paper faces precisely this settlement-governance problem. UDOS Reasoning Engine v7.4.9's `udos7/topology/market.py` gives a minimal but self-consistent internal market: tasks are first priced (`deposit`), agents bid (`Bid`: cost, self-assessed quality, load), the auction awards by "quality floor + value-for-money + load tie-break" (`TaskAuction.award`), after completion QA accepts (`settle(accepted=...)`), and only the unique accepted completer gets money; duplicate submissions, acceptance failures, and Byzantine results are all unpaid, and Byzantine results are additionally deducted from the deposit (`slash`). The whole ledger satisfies a conservation invariant.

Taking this mechanism apart, it in fact answers four plain questions. **Who can take?** — quality floor $\$q_{i\backslash ge\ f\$}$. **Given to whom?** — among the qualified, the one with the highest value-for-money $\$q_{i\backslash c_i\$}$ and lowest load. **On what basis paid after completion?** — paid only when QA accepts. **How much, how deducted?** — full reward given, duplicate/rejected/Byzantine given 0, Byzantine additionally deducted from deposit. These four questions sound like common sense, but writing them as code, as tests, as conservation assertions is engineering. All contributions of this paper are turning these four questions from "common sense" into "executable, testable, balanced-books" rules.

The boundary must first be declared. This is a market of **internal accounting units**, not a real cryptocurrency economy: no real pricing game, no Sybil defense, no cross-account encrypted settlement, and no linkage to any token/financialization. The question it answers is "inside a closed, auditable agent system, how to use accounting rules so resources are neither created nor destroyed out of thin air," not "how to design an open, Sybil-resistant economic protocol." Any industry narrative involving external "token economies" or "agent currencies" is an unverified reporting scope, and this paper does not cross-verify with it.

Specific contributions of this paper:

1. **Rules and invariants:** write the payment rule in mechanism-design language and prove conservation is a theorem (guaranteed by settlement code structure), whereas incentive compatibility is an empirical approximation (depends on acceptance hit rate).
2. **Reproducible tests:** 10 unit tests + matrix end-to-end integration, giving a measured "settlement rule $\times$ behavior type $\rightarrow$ conservation/incentive" matrix.

3. **Honest boundaries:** clarify what is theorem (conservation) and what is empirical (IC depends on acceptance hit rate), not inflating the engineering prototype into a mechanism-design theory contribution.

Of these three contributions, the third may be the most underrated. At a time when agent-system narratives are generally inflated, a paper honestly marking "this is internal accounting units, no Sybil defense, IC is only an empirical approximation" has more long-term value than a white paper claiming a "decentralized autonomous economy" — because the former knows its boundaries and the latter does not.

Choosing "conservation" rather than "optimal auction" as this paper's protagonist is a deliberate value orientation. In mechanism-design textbooks, VCG's elegant "truth-telling is optimal" conclusion is attractive, but engineering it needs public preference aggregation, the curse of dimensionality, and the computational cost of strategy-proofness; whereas a 300-line `market.py`, with a priority short-circuit and a conservation invariant, has solidly done the most basic thing of "money does not increase out of thin air." For a million-agent system, "balanced books" is more urgent than "optimal auction" — if the books do not balance, even the best auction is built on quicksand.

Boundary statement: all figures come from CPU deterministic simulation (`reports7/matrix_scale_demo.json`, `evidence_grade=cpu-proto`; unit-test figures come from `tests7/test_v749_market.py` and the `market.py` code), as **single-seed pilot point estimates**; the full-factorial simulation of strategy family $\times$ slashing rate $\times$ acceptance hit rate (≥ 1000 rounds/cell) has not been run, and the relevant positions keep [RESULT NEEDED].

2 Related Work

Task allocation and auctions have mature lines in multi-agent systems, distributed systems, and mechanism design. **Conceptually**, distributed task allocation was first formalized by the Contract Net Protocol: nodes contract tasks to the most capable through the three-stage "request for bids—bid—award" negotiation (Smith, 1980), which is precisely the historical source of UDOS's `TaskAuction` bidding/award. **Mechanically**, Vickrey showed the second-price sealed auction makes truth-telling a dominant strategy (Vickrey, 1961), and its generalization VCG simultaneously achieves incentive compatibility and Pareto efficiency under ideal assumptions; but Ausubel and Milgrom in their survey soberly point out that VCG is "theoretically elegant, practically lonely" — it is computationally hard, easily colluded, and does not satisfy budget balance, so it is rarely deployed at scale in real market design (Ausubel & Milgrom, 2006). Modern market design further extends the view to combinatorial auctions and procurement scenarios, discussing how award rules for "buyers buying services from multiple sellers" in electricity, spectrum, and procurement work under real constraints (Milgrom, 2019). **On evidence**, Byzantine fault tolerance on the distributed-systems side has a mature state-machine replication tradition: PBFT tolerates $f < n/3$ Byzantine nodes under asynchronous networks, with implementation overhead only about 3% more than an unreplicated system (Castro & Liskov, 1999); and "slashing" as a Byzantine economic penalty has extensive separate discussion in blockchain/staking literature [CITATION NEEDED: slashing Proof of Stake]. **Difference from UDOS:** the above works are mostly at the open economy or consensus layer and do not solve "how the settlement ledger balances in a closed internal market"; Sybil resistance in the open economy is even harder [CITATION NEEDED: Sybil attack Douceur].

Closest to this paper is the branch of "multi-agent task allocation with payment" [CITATION NEEDED: multi-agent task allocation payment; market-based multi-robot task allocation; peer prediction

mechanism proper scoring]. Such works usually use auctions to allocate tasks to the lowest-cost or highest-utility agents. Our difference is not in auction theory but in the **engineering conservation of the settlement ledger**: most auction works care about "who wins," and this paper cares about "how to pay after winning, how to ensure money is neither added nor lost out of thin air, and how to make duplicate labor and false reporting unprofitable on the ledger." In other words, we step back from VCG's "truth-telling dominance" and replace it with the engineering-auditable "pay only on acceptance, pay only the unique completer, Byzantine must be slashed" — a pragmatic response to the VCG practical defects Ausubel–Milgrom pointed out.

Our positioning is **mechanism engineering + simulation reference implementation**, not claiming VCG-style theoretical optimality. The reason: in a closed internal market, fully IC complex mechanisms (VCG needs public preference aggregation and is computationally expensive) are not worthwhile; a simple "quality floor + value-for-money + acceptance payment + slashing" is engineering-auditable and testable, already sufficient to suppress duplicate labor and false reporting. When theoretical depth is insufficient, we honestly position it as an empirical paper.

This paper must also be clearly separated from popular narratives like "token economies/agent currencies." The latter often advocates "issuing coins to agents and letting the market self-regulate," but that is an open-economy problem needing Sybil defense, price discovery, and external trust anchors, far beyond this paper's scope. Between these two there is a gap this paper fills: **most auction works care about "who wins," and few write "how to settle after winning, how to ensure conservation, and how to make duplicate labor and false reporting unprofitable" as a testable-invariant engineering reference implementation**. Academia has beautiful theorems for VCG's incentive compatibility but insufficient attention to the engineering problem of "a 300-line internal ledger covered by pytest with balanced books." This paper does not pursue VCG-style elegance but provides a settlement reference implementation directly readable, runnable, and extensible, letting later work do full-factorial strategy simulation on it rather than writing a ledger from scratch.

It must be admitted that the cost of this engineering orientation is limited theoretical depth: we do not prove our auction is optimal, do not prove our slashing mechanism is collusion-resistant, and do not prove IC holds over a general utility domain. All conclusions of this paper are bounded within the specific engineering configuration of "internal accounting units, oracle acceptance, single-thread sequential settlement." Treating it as a mechanism-design theory contribution is a misreading; treating it as a reproducible engineering benchmark is this paper's positioning.

3 Problem Definition and Assumptions

3.1 Notation and rules

A task t has budget B_t (injected by `deposit`). Agent i bids (c_i, q_i, ℓ_i) : cost c_i , self-assessed quality $q_i \in [0, 1]$, load ℓ_i . Auction floor f . Award rule:

$$\text{eligible} = \{i: q_i \geq f\}, \quad \text{winner} = \arg \max_{i \in \text{eligible}} \left(\frac{q_i}{\max(c_i, 10^{-9})}, -\ell_i \right).$$

i.e. first pass the quality floor, then award by value-for-money q/c , with load as the order tie-break (same value-for-money choose the lower load).

The settlement rule (`settle`) is a priority short-circuit:

1. If the task has already been paid (`task_id` in `paid_tasks`) $\rightarrow$ pay 0, reason `already_paid`;
2. If marked duplicate labor (`duplicate=True`) $\rightarrow$ pay 0, reason `duplicate_work`;

3. If acceptance fails (`accepted=False`) → pay 0, reason `rejected`, and deduct \$\$\$ from that agent's balance (slashing);
4. Otherwise (accepted, not paid, not duplicate) → pay `$B_t$` to that agent and record in `paid_tasks`.

3.2 Falsifiable assumptions

- **H1 (duplicate-labor payment is 0):** the unique-payment constraint makes any duplicate/repeated submission's payment 0. Falsifiable: either `test_only_accepted_unique_work_paid / test_duplicate_work_flag_not_paid` fails.
- **H2 (slashing makes Byzantine expected return negative):** under a given slashing rate \$\$\$ and acceptance hit rate \$p\$, the Byzantine strategy's expected return < 0 . Falsifiable: when \$\$\$ is small enough and acceptance hit rate high enough, Byzantine expected return ≥ 0 .
- **H3 (quality floor prevents low-price low-quality):** the floor \$f\$ filters out bids with $q_i < f$. Falsifiable: `test_quality_floor_filters_cheap_low_quality` fails.
- **H4 (budget conservation):** `total_paid` $\leq$ `total_budget` and `balance_sum` = `total_paid` - `slashed`. Falsifiable: `conservation_check().conserved` is False.

There is also an implicit assumption worth making explicit: **H5 (acceptance is an oracle)**. Current QA judges "whether the result is qualified" as a known Boolean, with acceptance hit rate $p=1$. This means this paper's IC conclusions hold under the ideal acceptance of "false reporting is always caught"; once acceptance has misses, H2 must be recomputed by the formula in Section 4.5. We do not hide this assumption but list it as H5, letting readers see at a glance the conditions under which IC holds.

4 Method and System Design

4.1 The auction: value-for-money award

The eligible filtering and value-for-money sorting of `TaskAuction.award` are two lines of code, but the design intent is worth clarifying. Why use q/c rather than pure lowest cost? Because pure lowest cost incentivizes "low-price low-quality" — whoever bids lowest wins, and quality naturally degrades. Why add the quality floor f rather than pure value-for-money? Because value-for-money q/c can still be high when q is very small (c extremely small), and the floor first keeps out "not qualified at all" bidders, then selects the best value-for-money among the qualified. Load ℓ_i is the order tie-break: when two bids have the same value-for-money, choose the currently lower-loaded one, avoiding pressing all tasks onto the same agent.

In Table 2 unit tests, `test_auction_picks_best_quality_cost_ratio` directly demonstrates this: three bidders $a(\text{cost}=8, q=0.8)$, $b(\text{cost}=4, q=0.8)$, $c(\text{cost}=3, q=0.2)$. a and b have the same quality 0.8, b has lower cost, so b 's value-for-money beats a ; c has the lowest cost but quality 0.2 is treated as low quality. Finally b wins. This shows q/c simultaneously penalizes "too expensive" and "too poor," rather than only looking at price.

4.2 The settlement ledger: priority short-circuit

The priority short-circuit of `ContributionLedger.settle` is the core of this mechanism. It first asks "has this task been paid," then "is it duplicate labor," then "did acceptance pass," and finally pays. Each short-circuit layer corresponds to a defense against one strategic behavior:

- **Already-paid check** prevents "a task submitted by two agents, paid twice";

- **Duplicate marking** prevents "the same agent repeatedly submitting the same thing to inflate workload";
- **Acceptance check** prevents "getting money for an unqualified result";
- **Slashing** additionally deducts deposit on acceptance failure, raising the expected cost of false reporting.

4.3 The conservation invariant

`conservation_check()` computes:

$$\text{\$}\ \text{\textit{total_budget}} = \sum_t B_t, \text{\textit{balance_sum}} = \sum_i \text{\textit{bal}}_i, \text{\tag{2}} \text{\$}$$

$$\text{\$}\ \text{\textit{holds}} = \text{\textit{total_budget}} - \text{\textit{total_paid}}, \text{\tag{3}} \text{\$}$$

$$\text{\$}\ \text{\textit{conserved}} = \left(\text{\textit{total_paid}} \leq \text{\textit{total_budget}} + \text{\textit{varepsilon}} \right) \wedge \left(\text{\textit{balance_sum}} - (\text{\textit{total_paid}} - \text{\textit{slashed}}) \leq \text{\textit{varepsilon}} \right), \text{\tag{4}} \text{\$}$$

Where $\text{\textit{varepsilon}} = 10^{-9}$ tolerates floating-point error. This invariant is a **theorem, not empirical**: because `settle` only transfers B_t from the budget pool into some agent's balance when acceptance passes ($\text{\textit{total_paid}} += B_t$, $\text{\textit{balances}}[\text{\textit{agent}}] += B_t$), and slashing only deducts from balance ($\text{\textit{balances}}[\text{\textit{agent}}] -= s$, $\text{\textit{slashed}} += s$), with no code path creating balance out of thin air, the ledger identity always holds arithmetically. What really needs testing is "whether the rules trigger as designed," not "whether the arithmetic conserves."

Worth expanding is "why conservation deserves a dedicated proof." In engineering practice, the most common form of ledger bug is not "money computed wrong" but "money whereabouts unknown": a payment made but not recorded, a slash deducting balance but not recording slashed, a task paid twice. These bugs in small systems are found by human reconciliation but in million-scale systems are dark ledgers. Writing `balance_sum = total_paid - slashed` as one executable `conservation_check()` and calling it once at the end of each integration test automates "reconciliation" into a CI gate: any payment with unknown whereabouts immediately makes `conserved` False. This practice of "turning the accounting identity into a test assertion" is much cheaper than post-hoc auditing.

4.4 The boundary between theorem and empirical

Two kinds of conclusions must be strictly separated. **Conservation is a theorem**: as long as `settle` is not bypassed and no floating-point drift appears, `conserved` is always true. **Incentive compatibility (IC) is an empirical approximation**: whether a Byzantine strategy has negative return depends on the combination of "acceptance hit rate $\$p\$$ " and "slashing rate $\$s\$$ " — if acceptance is too loose ($\$p\$$ low) or slashing too light ($\$s\$$ small), Byzantine expected return may turn positive. This paper currently tests only one fixed configuration ($\$s=1.0\$$, strict acceptance), and the full-factorial threshold grid is to be supplemented ([RESULT NEEDED]).

4.5 Conditions for individual rationality and incentive compatibility

Putting this rule set into mechanism-design language can more precisely state what it promises and does not promise.

Individual rationality (IR): an honest agent's expected utility of participating in the market is non-negative. Because payment is made only on acceptance and is the task's full reward, the honest completer's net return is positive (reward minus true cost, worth doing as long as $\text{reward} > \text{cost}$); not participating gives 0. Therefore under the task-pricing premise of " $\text{reward} \geq \text{true cost}$," honest agents

have motive to participate. But this relies on task pricing itself being reasonable — if reward is set below true cost, even honest agents will not take it, and IR is broken. This paper assumes reasonable pricing as an external premise and does not solve it within the market mechanism.

Incentive compatibility (IC): is honest bidding better than false reporting? In this mechanism, IC is **partial and empirical**. Duplicate labor is blocked by the unique-payment constraint (no matter how repeated, the second and later submissions are paid 0), and this part of IC is structural. But whether "low-quality bid rushing" is suppressed depends on the quality floor q_f and acceptance hit rate p : if self-assessed quality q_i can be falsely reported (agent self-assesses 0.9 but actually delivers 0.3), the floor is bypassed and only acceptance can backstop; the lower the acceptance hit rate p , the higher the probability low-quality deliveries slip through. Byzantine slashing s further raises the expected cost of false reporting, but s must be greater than "the expected return of false reporting passing acceptance" to have deterrence.

The Byzantine expected utility can be written as:

$$E[u_{\text{byz}}] = (1-p)R - p s, \tag{5}$$

Where R is the illegal return when false reporting slips acceptance, p is the acceptance hit rate, and s is slashing. When $(1-p)R < p s$, i.e. $s > R(1-p)/p$, the Byzantine expected return is negative. This is the analytical form of the "slashing-rate break-even point" and the threshold the [RESULT NEEDED] full-factorial simulation verifies. This paper does not hand-compute this threshold without measured R , p .

5 Experimental Setup

Per the v2 specification, the empirical chapter unfolds as "experimental setup → comparison (unit-test ledger vs matrix clean/stressed) → sensitivity (analytical thresholds of slashing rate/acceptance hit rate) → validity threats (Chapter 8)."

5.1 Unit-test protocol

`tests7/test_v749_market.py` contains 10 tests, all deterministic, CPU, no randomness:

1. Value-for-money award (same-quality low-cost wins);
2. Quality floor filters low-price low-quality;
3. No eligible bidder, no winner;
4. Load breaks ties;
5. Only the unique accepted completion paid (duplicate completion unpaid);
6. Duplicate-labor marking unpaid;
7. Byzantine result unpaid and slashable;
8. Conservation invariant (mixed 3 tasks);
9. Payment never exceeds budget;
10. Rejected work recorded.

5.2 End-to-end integration

`scripts7/matrix_demo.py` runs the agent matrix of 24 work orders, in two levels clean (no faults) and stressed (3 faulty nodes injected), with market settlement called by `udos7/topology/matrix.py`. Outputs `reports7/matrix_scale_demo.json` (evidence_grade=cpu-proto). Hardware: CPU, deterministic simulation, agents not real LLMs.

It must be noted that the market budget in matrix integration is 240 (24 work orders $\times$ 10/order), while the unit-test mixed 3-task budget is 24 ($T1=10+T2=8+T3=6$). The two are not the same set of figures: the unit test is the minimal reproducible conservation demo, and matrix integration is the 24-order scale demo. Both clean/stressed levels pay out all 240 ($\text{total_paid}=240$) because all work orders are ultimately accepted; the unit test deliberately keeps an acceptance-failing T2 so $\text{holds}=8>0$, demonstrating the non-trivial case of "unpaid budget on hold." If all work orders smoothly accept, holds is always 0 and conservation degenerates to $\text{total_paid}=\text{total_budget}$ — this trivial case cannot test the holds field, so the unit test specifically constructs T2 failure.

6 Results

6.1 Unit-test conservation: mixed 3 tasks

Table 1 reproduces the ledger of `test_conservation_invariant`. Three tasks: $T1=10$ (paid to a), $T2=8$ (acceptance rejected, slash 1.0), $T3=6$ (paid to c).

Table 1 Mixed 3-task unit-test ledger (verified, `test_conservation_invariant`)

Quantity	Value	Note
<code>total_budget</code>	24	$T1+T2+T3$
<code>total_paid</code>	16	$T1(10)+T3(6)$
<code>slashed</code>	1	T2 slash
<code>holds</code>	8	T2 unpaid budget still in pool
<code>balance_sum</code>	15	$=16 \text{ paid} - 1 \text{ slashed}$
<code>conserved</code>	true	invariant holds

Table 1 note: evidence grade verified (unit-test hardcoded assertion). Both sides of Eq.(4) hold: $\text{total_paid}=16 \leq \text{total_budget}=24+\epsilon$; $\text{balance_sum}=15=(\text{total_paid } 16)-(\text{slashed } 1)$, error $<\epsilon$. $\text{holds}=8$ is Eq.(3), the T2 unpaid budget on hold.

`balance_sum=15` exactly equals `total_paid(16) - slashed(1)`, verifying the conservation invariant. T2 acceptance failed and was unpaid, and its 8 budget stays in `holds` (the pool), neither disappearing nor being transferred to others out of thin air. Figure 1 places this ledger alongside the matrix clean/stressed levels.

Here the accounting meaning of `holds=8` needs explanation. Total budget 24 = paid 16 + unpaid holds 8. The unpaid 8 is the budget for T2 rejected by acceptance; it was neither paid to a/b/c (which would cause overspend or mispayment) nor disappeared from the system (which would break conservation), but is "on hold" in the pool awaiting reallocation — which is precisely the role of the `holds` field: tracking budget "still pending, not yet transferred to anyone." This design keeps the ledger always closed: total budget issued = paid to agents + slashed back + still in pool. At any moment auditing, `total_budget` can be split into these three items, with no "dark ledger."

Figure 1: Three-ledger conservation comparison

Figure 1 Three-ledger conservation comparison. Left column: unit-test mixed 3 tasks (budget 24 = paid 16 + slashed 1 + holds 8; `balance_sum` 15); middle column: matrix clean level (budget 240 = paid 240 + holds 0); right column: matrix stressed level (budget 240 = paid 240 + holds 0). Each level is

conserved=true. Evidence grade: unit test verified, matrix cpu-proto. Data source reports7/matrix_scale_demo.json + tests7/test_v749_market.py.

6.2 Settlement rule × behavior type

Table 2 (Figure 2) is the decision matrix of "settlement rule × agent behavior → payment result." This is the direct correspondence of H1/H2/H3.

Table 2 Settlement rule × behavior type (verified, from market.py settle logic)

Agent behavior	Paid?	Reason	Slashed?
Honest submission, accepted	Full reward	accepted	No
Repeated submission of same task	0	already_paid / duplicate_work	No
Low-quality submission, rejected	0	rejected	Optional (slash>0)
Byzantine result, rejected	0	rejected	Yes (deduct deposit)
No eligible bidder	No winner	—	—

Table 2 note: evidence grade verified (settle priority short-circuit logic + 10 unit tests). Reading: row by row "which short-circuit this behavior falls into → how much paid → whether slashed," i.e. the ledger result of each strategic behavior. The strict conclusion that Byzantine "expected return is negative" needs full-factorial simulation (Eq.(5)), currently [RESULT NEEDED].

Figure 2: Settlement rule × behavior type

Figure 2 Settlement rule × behavior type decision matrix. The horizontal axis is agent behavior (honest/duplicate/low-quality/Byzantine/no eligible bid), and the vertical axis is the settlement short-circuit level (already-paid check → duplicate marking → acceptance check → payment), converging to payment result (full/0/no winner) and whether slashed. Evidence grade verified; the sign of Byzantine expected return is [RESULT NEEDED].

H1 holds: honest submission pays full, and the other three strategic behaviors all pay 0. H3 holds: the quality floor blocks $q < f$ at the award stage (test_quality_floor), without waiting for acceptance. H2 partially holds: Byzantine payment is 0 and slashed, but the strict conclusion "expected return negative" needs full-factorial simulation — currently negative balance is observed only at the single point $s=1.0$, strict acceptance, [RESULT NEEDED: slashing rate {0,0.5,1,2} × acceptance hit rate {0.7,0.9,1.0} grid].

Specifically in test_rejected_byzantine_work_unpaid_and_slashable: Byzantine agent z starts with balance 3.0, and after acceptance rejection settle(accepted=False, slash=2.0), z's balance falls to 1.0, slashed=2.0, total_paid=0. This means z not only got no payment but also lost 2.0 deposit — under this configuration, the ledger cost of false reporting is certain. But this is one deterministic demo, not an expected-return statistic; whether z will falsely report depends on its estimate of "the probability p of being caught by acceptance," and this p under current oracle acceptance is always 1 (a bad result is always caught once submitted), so z must lose. Once acceptance has misses ($p < 1$), the conclusion must be recomputed by the formula in Section 4.5.

Two other award details worth recording: test_load_breaks_tie verifies same value-for-money chooses the lower-loaded (load=1 beats load=5), avoiding task clustering; test_no_eligible_bid_no_winner verifies no forced award when no one is qualified (winner stays None), a conservative design of "rather leave empty than force," safer than "randomly pick one."

6.3 End-to-end integration: clean vs stressed

Table 3 and Figure 3 give the two-level comparison of matrix integration.

Table 3 Matrix end-to-end settlement (cpu-proto, 24 work orders; source reports7/matrix_scale_demo.json)

Metric	clean	stressed (3 faulty nodes)
n_orders	24	24
accepted	24	24
qa_rounds	24	25
retries	0	5
isolated_agents	0	3
total_budget	240	240
total_paid	240	240
slashed	0	0
balance_sum	240	240
conserved	true	true
duplicate_work	0	0

Table 3 note: evidence grade cpu-proto (single-seed pilot). Both levels total_paid=240=total_budget, so holds=0, balance_sum=240, and Eq.(4) always holds. The stressed level injects faulty nodes `spec-kinematics-0/spec-spatial-1/spec-dataflywheel-2`, isolated by the circuit breaker (isolated_agents=3) producing 5 reworks (retries=5, qa_rounds=25), but `duplicate_work=0` — rework is redoing the same work order, and the unique-payment constraint guarantees no duplicate payment. The stressed level `slashed=0` is because this run's faults take the circuit-breaker path rather than the market-slash path, not that the slashing mechanism is missing.

Figure 3: clean vs stressed

Figure 3 Matrix end-to-end clean/stressed two-level comparison. Left clean, right stressed; the top row is governance/settlement metrics (n_orders, accepted, retries, isolated_agents), and the bottom row is the ledger (budget/paid/slashed/balance_sum/conserved). Both levels are `conserved=true`, 24/24 settled by acceptance. Evidence grade cpu-proto (single-seed pilot). Data source `reports7/matrix_scale_demo.json`.

Both levels are 24/24 settled by acceptance, `conserved=true`. The stressed level injected 3 faulty nodes (`spec-kinematics-0`, `spec-spatial-1`, `spec-dataflywheel-2`), and the system isolated them through the circuit breaker (isolated_agents=3), producing 5 reworks (retries=5, qa_rounds=25), but **rework produces no duplicate payment** — `duplicate_work=0`, because `settle`'s unique-payment constraint guarantees a work order is paid only once. Note: the stressed level `slashed=0`, because this integration's fault governance takes the **circuit-breaker isolation** rather than **market slashing**; the slashing mechanism itself is verified in unit tests (Tables 1, 2). These are two independent defenses: market slashing handles "falsely reported but completed," and circuit-breaker isolation handles "node broken and unable to complete at all."

From governance metrics, the clean level's governance is all zero (state_loss=0, duplicate_work=0, no_closer=0, premature_completion=0, unbounded=0, trace_gaps=0), trace_verified=true, trace_len=48; the stressed level likewise governance ok, trace_verified=true, trace_len=50. That is, even

with 3 faulty nodes injected and 5 reworks produced, the governance audit found no duplicate labor, state loss, or trace break. This confirms the "unique-payment constraint" also works in end-to-end integration: rework is redoing the same work order and will not pay a second time because of redoing.

7 Discussion

Why conservation is a theorem while IC is empirical. This distinction matters because it determines what we can promise. Conservation: as long as the code path does not bypass `settle`, arithmetic identity — this is promise-able. IC: whether Byzantine has negative return depends on the combination of acceptance hit rate and slashing rate — this can only be measured in a specific configuration. This paper's current slashing configuration ($s=1.0$ on T2) makes Byzantine agent z's balance fall from 3.0 to 1.0 (`test_rejected_byzantine_work_unpaid_and_slashable`), negative return on the ledger; but if s is set to 0, Byzantine only loses time, not deposit, and expected return may turn positive. Therefore this paper **does not promise** "Byzantine has negative return at any slashing rate," only promises "negative balance observed under the configuration $s=1.0$, strict acceptance."

There is also a design choice worth expanding: why slashing goes through deposit deduction rather than directly confiscating all balance? `settle`'s `slash` is a configurable parameter (default 0, passed by the caller), meaning slashing strength is a policy knob, not hardcoded. In unit tests we use two strengths `slash=1.0` (T2) and `slash=2.0` (z) to demonstrate: the former is "minor false reporting," the latter "explicit Byzantine." Making strength a parameter is because different tasks have different harm degrees — a task writing a wrong document and a task forging a safety-critical result should be slashed by different multiples. This paper does not prescribe the optimal slashing rate; that is the question the Section 4.5 threshold grid answers.

There is also a design philosophy worth recording: this mechanism **rather leaves empty than mispays**. `test_no_eligible_bid_no_winner` makes clear that when all bidders fail the quality floor, the auction returns None and does not force. This is opposite to the aggressive scheduling of "must dispatch the task before deadline." In an internal market, forcing to an unqualified bidder ultimately only produces rework and duplicate payment; rather let the task temporarily be unclaimed, await the next bidding round or escalate to a superior, than mispay. This "conservative-first" default, with P5's "rather a wider interval" and P6's "rather say unidentifiable," is the same engineering ethics: choose the conservative side under uncertainty.

8 Limitations and Validity Threats (separate chapter)

T1 Internal threat (main). Unit tests are deterministic single points, without ≥ 1000 -round strategy simulation, and IC conclusions are narrow. **Most likely counterexample:** changing slashing rate s or acceptance hit rate p , Byzantine expected return turns positive. **How this paper guards:** write the Eq. (5) break-even point $s > R(1-p)/p$ as an analytical threshold, with the full-factorial grid marked [RESULT NEEDED] .

T2 Construction threat: acceptance is an oracle. Current QA judges "whether the result is qualified" as a known Boolean (acceptance hit rate $p=1$), and in a real system acceptance itself may be manipulated — a deeper governance problem this paper does not solve.

T3 External threat. CPU prototype, agents not real LLMs, million-scale only closed-form extrapolated, not run in a real distributed system.

T4 Measurement threat. The stressed level `slashed=0` does not mean the slashing mechanism is ineffective, only that this run did not trigger the slash path (faults took the circuit-breaker isolation path).

T5 Concurrency threat. `ContributionLedger` uses a normal Python `dict`, assuming single-thread sequential settlement. In real distributed deployment, if two agents almost simultaneously submit the same work-order result, the single-thread dict may race: both `settle`s read "not paid" and then both pay, breaking conservation. Currently this is avoided by the caller (`matrix.py`) single-thread serially calling `settle`, rather than the ledger itself being concurrency-safe. Conservation under "sequential settlement" is a theorem, and under "concurrent settlement" needs linearizability guarantees, which this paper does not solve.

T6 No Sybil defense. "Unique payment" relies on `paid_tasks`'s task→agent mapping, assuming one task has only one owner; it **does not defend Sybil** — if an attacker bids with multiple identities and repeatedly submits, the `duplicate` marking triggers only when upstream correctly identifies "this is duplicate labor." This paper honestly declares: no Sybil defense, no identity deposit, no encrypted settlement.

9 Resource Gates and Applicability Boundaries

- This paper's conclusions stop at cpu-proto (CPU deterministic prototype). Full-factorial strategy simulation (≥ 1000 rounds/cell), real LLM agent bidding, and real distributed ledgers are all to be supplemented.
- No linkage to any real crypto/token economy, no financialization claim.
- Applicability boundary: this paper's method applies to "closed, auditable, internal-accounting" multi-agent task markets; it does not apply to open economies, scenarios needing Sybil defense, or real-currency settlement.
- Relation to P1/P2/P3: P1 covers topology reducing complexity, P2 covers BFT stop decisions, P3 covers fault-governance RCT, and this paper covers how money settles in the governance loop — the four together form a complete agent-matrix governance stack.

Specifically, the resource gate's limitation on P11 conclusions appears in three places. First, **strategy simulation not run**: currently only deterministic unit tests and end-to-end two levels, without the expected-return distributions of five strategies "honest/duplicate/low-quality rushing/Byzantine/collusive price suppression" under different slashing rates and acceptance hit rates, so H2's IC conclusion can only stop at a single point. Second, **acceptance is an oracle**: current QA rules are clear and acceptance hit rate is always 1 (bad results always caught), and in a real system acceptance itself may be manipulated or miss, a deeper governance problem. Third, **scale not measured**: million-agent only closed-form extrapolated (see P1), and the settlement ledger's consistency under real distributed concurrency (such as the race of two agents simultaneously submitting the same work order) needs distributed-ledger mechanisms, which this paper's single-thread `dict` implementation does not handle.

10 Conclusion

On UDOS v7.4.9's internal task market, we give a settlement mechanism of "quality floor + value-for-money award + payment only to the unique accepted completer + Byzantine slashing." The conservation invariant `balance_sum = total_paid - slashed` is a theorem, and both the mixed 3 tasks and matrix clean/stressed levels measured `conserved=true`; duplicate labor and false reporting pay 0 on the ledger, and all 10 unit tests pass. The honest part: IC is an empirical approximation, not a theorem, and the full-

factorial threshold grid of slashing rate/acceptance hit rate has not run; no Sybil defense, no real pricing game. We write separately what is promise-able (conservation) and what can only be approximated (IC), and mark unfinished work with [RESULT NEEDED]. An auditable internal market is first a market with balanced books.

There are three follow-up works. First, run full-factorial strategy simulation (strategy family {honest, duplicate, low-quality rushing, Byzantine, collusive price suppression} $\times$ slashing rate {0,0.5,1,2} $\times$ acceptance hit rate {0.7,0.9,1.0}, ≥ 1000 rounds/cell), locate the slashing-rate break-even point $s^* = R(1-p)/p$, and upgrade H2 from a single point to an extrapolatable threshold curve. Second, connect the settlement ledger to real LLM agent bidding and test "whether self-assessed quality q_i can be honestly reported" — the most fragile link of IC. Third, handle concurrency races: the current single-thread dict implementation does not handle the race of "two agents simultaneously submitting the same work order," and distributed deployment needs a linearizability ledger backend. Reading three papers together (P5 credibility + P6 identifiability + P11 settlement) jointly points to an engineering philosophy: an honest system is not one that can do everything, but one that knows what it cannot do* — in settlement, knowing which payment should be made, which must be deducted, and which should rather be on hold than mispaid.

Finally returning to this paper's plainest claim: in the process of multi-agent systems moving toward million scale, "balanced books" is a more basic and scarcer capability than "optimal mechanism." This paper proves this with 300 lines of code and 10 tests. It is not elegant, not complex, not theoretical, even a bit boring — but a non-boring, error-free, auditable internal ledger is precisely the foundation supporting all intelligent collaboration above. A foundation need not be pretty; it needs to bear weight. May this honest 300-line ledger become a directly readable, modifiable, runnable starting point for later people discussing settlement on million-agent systems, rather than yet another white paper no one can reproduce.

References

Note: the verified-reference fields already cited in this draft are copied from the whole-volume "Master Reference Library" (verification date 2026-09-19); the rest are to-be-verified search expressions, not treated as cited facts, and before submission metadata and DOI must be verified item by item through academic search.

Verified citations (✓ master library 2026-09-19)

- Smith, R. G. (1980). The Contract Net Protocol: High-Level Communication and Task Control in a Distributed Problem Solver. IEEE Transactions on Computers, C-29(12). DOI:10.1109/TC.1980.1675516. — Request-bid-award negotiation, the historical source of UDOS task dispatch and TaskAuction.
- Vickrey, W. (1961). Counterspeculation, Auctions, and Competitive Sealed Tenders. Journal of Finance, 16(1). — Second-price auction = VCG prototype, truth-telling as dominant strategy.
- Ausubel, L. M., & Milgrom, P. (2006). The Lovely but Lonely Vickrey Auction. In Combinatorial Auctions (book chapter), MIT Press. — VCG theoretical beauty and practical defects (collusion/budget imbalance/computational hardness), the direct basis for this paper's "pragmatic step back."
- Milgrom, P. (2019). Auction Market Design: Recent Innovations. Annual Review of Economics, 11. — Modern market design for combinatorial auctions/electricity/spectrum/procurement, procurement-auction paradigm reference.

- Castro, M., & Liskov, B. (1999). Practical Byzantine Fault Tolerance. OSDI 1999 (USENIX). — Tolerates $\frac{n}{3}$ Byzantine nodes under asynchrony; the distributed-systems comparison for this paper's Byzantine slashing.

To-be-verified reference search expressions and candidate directions

- Contract net/task allocation: multi-robot task allocation market-based; multi-agent task allocation payment mechanism design.
- Auction theory: procurement reverse auction; peer prediction mechanism proper scoring rule (core peer-prediction literature to supplement).
- Byzantine/slashing: slashing Proof of Stake economic penalty.
- Sybil resistance: Sybil attack Douceur; Sybil resistance open economy.

Appendix A Reproduction commands and test index

```
cd release-v5.4.4/udos-engine          # main, tag v7.5.0, commit 1a71270
pytest tests7/test_v749_market.py -q  # 10 unit tests
python3 scripts7/matrix_demo.py       # end-to-end integration -> reports7/matrix_scale_demo.json
```

Key source: `udos7/topology/market.py` (Bid / TaskAuction / ContributionLedger), `udos7/topology/matrix.py` (integration), `tests7/test_v749_market.py`.

Suggested reproduction order: first run `pytest tests7/test_v749_market.py -q` to see all 10 unit tests green (the minimal verification of conservation and rules), then run `matrix_demo.py` to see end-to-end two levels `conserved=true`. Unit tests are deterministic and complete in seconds; matrix integration is slightly slower but still CPU. Any step with `conserved=false` means the ledger was broken — which is precisely the meaning of `conservation_check()` as a CI gate.

Appendix B Evidence ledger

Figure	Source	Grade
total_paid=16/holds=8/balance_sum=15/ slashed=1	<code>tests7/test_v749_market.py:</code> <code>test_conservation_invariant</code>	verified (unit test)
Matrix clean/stressed conserved=true	<code>matrix_scale_demo.json: clean/stressed.market</code>	cpu-proto
24/24 accepted, retries 0/5, isolated 0/3	<code>matrix_scale_demo.json</code>	cpu-proto
10 unit tests	<code>tests7/test_v749_market.py</code>	verified
External token-economy figures	—	unverified

Final reminder: all "meets standard"/"passes" wording in this paper is bounded within the specific scope of "CPU deterministic prototype, single-thread sequential settlement, oracle acceptance." It is not a universal conclusion that "mechanism design holds in any multi-agent system," but "on this specific system of 300-line `market.py`, 10 pytest, 24-order matrix integration, settlement conservation is guaranteed by code structure." Any extension beyond this scope needs new experiments and new evidence.

[RESULT NEEDED]: slashing-rate $\times$ acceptance-hit-rate full-factorial ≥ 1000 -round simulation, expected-return CI for each strategy, false-slash rate (probability honest agents are slashed). [CITATION NEEDED]: the five candidate directions in Section 2.

Supplementary note: the "unit-test" figures in the table (total_paid=16 etc.) come from `test_conservation_invariant`'s hardcoded assertions and are deterministically recomputable; the "matrix clean/stressed" figures come from `matrix_scale_demo.json` and are cpu-proto prototype run results; the two have different evidence grades and should not be mixed. The stressed level `slashed=0` does not mean the slashing mechanism is missing, but that this run's faults take the circuit-breaker path — this distinction is honestly stated in Section 6.3.

Appendix C Internal review record (pre-submission five-dimension self-assessment, look only, do not change)

Scored on five dimensions per `doubao-academic-evaluator` (0–10, cpu-proto prototype scope):

1. **Problem importance:** 5. Internal-market settlement indeed matters as agent scale grows, but this paper positions an engineering reference implementation, not a theoretical breakthrough.
2. **Technical correctness:** 7. The conservation invariant is guaranteed by code structure with complete test coverage; deductions for IC only at a single point, no full-factorial simulation.
3. **Experimental rigor:** 5. 10 unit tests + end-to-end two levels are hard evidence; but strategy-family simulation not run, IC conclusions narrow.
4. **Novelty:** 5. Mechanism components classical (auction/slashing/ledger), novelty in engineering conservation tests and honest-boundary declaration.
5. **Reproducibility:** 9. Scripts, tests, report JSON complete, CPU rerunnable.

Fatal-defect check: the conservation vs IC boundary is honestly distinguished; no fabricated p-values/CI/DOI; no Sybil defense declared. **Conclusion:** currently a submittable empirical-paper draft; before submission full-factorial strategy simulation must be added to upgrade H2 from a single point to a threshold grid, otherwise only workshop/engineering tracks.

Author's intended-use statement

- **Target journals/conferences:** candidates AAMAS / AAMAS workshop; engineering-leaning can submit ICWS, JSS. If theoretically deepened can submit EC. **Quartile/IF [to verify]**.
- **Pre-registration plan:** the conservation invariant as a hard constraint (already in code); the factorial design and stopping rules of the slashing-rate $\times$ acceptance-hit-rate grid registered before submission.
- **Data and code availability:** Apache-2.0; tag `v7.5.0`, commit `1a71270`; reports `reports7/matrix_scale_demo.json`, `tests7/test_v749_market.py`.
- **AI-use statement:** the author led mechanism design and conclusions, AI assisted structured writing, figures, and language polishing; figures sourced by the author against source and tests, AI produced no experimental data.